

PRM3

Théorie : Leçon 2

Java :

Expressions booléennes et instruction
alternative

Booléens

Une variable de type `boolean` vaut soit vrai (`true`) soit faux (`false`).

```
boolean ok = true;
```

Opérateurs relationnels

Ils servent à comparer deux expressions compatibles.

$==$: est égal (**ne pas confondre avec $=$**)

$!=$: est différent

$<$: est strictement plus petit

$<=$: est plus petit ou égal

$>$: est strictement plus grand

$>=$: est plus grand ou égal

Expressions booléennes

Les expressions composées à l'aide des opérateurs relationnels ont une valeur de vérité (`true/false`).

Exemples :

<code>(1 + 2) == 3</code>	dénote <code>true</code>
<code>2 >= 3</code>	dénote <code>false</code>
<code>5 % 2 == 0</code>	dénote <code>false</code>
<code>5 != 5</code>	dénote <code>false</code>

On appelle *condition* une expression à valeur booléenne.

Opérateurs logiques

Ces opérateurs permettent de composer des expressions booléennes :

& & : conjonction

| | : disjonction

! : négation

a	! a
true	false
false	true

a	b	a && b	a b
true	true	true	true
true	false	false	true
false	true	false	true
false	false	false	false

Opérateurs logiques

`a && b` est vrai si et seulement si `a` est vrai et `b` est vrai

`a || b` est faux si et seulement si `a` est faux et `b` est faux

a	b	a && b	a b
true	true	true	true
true	false	false	true
false	true	false	true
false	false	false	false

Expressions booléennes

`1 > 2 && 2 > 3`

`2.3 < 4 || 2.3 > 10`

`!(3 == 4)`

`(x > 0 && x < 100) || (x < 0 && x > -100)`

Remarque syntaxique :

`a < b < c` n'est pas correct

`a < b && b < c` est correct

Loi de De Morgan

! (a && b) est équivalent à (!a) || (!b)

! (a || b) est équivalent à (!a) && (!b)

Ces lois permettent, par exemple, d'augmenter la lisibilité de certaines expressions.

Exemple d'expressions équivalentes :

```
! (0 >= x || x >=1000)
```

```
0 < x && x < 1000           // plus lisible
```


Evaluation paresseuse

Les opérateurs logiques `&&` et `||` effectuent une **évaluation paresseuse** de leurs arguments.

L'évaluation des arguments se fait **de gauche à droite** et ne sont évalués que les arguments strictement nécessaires à la détermination de la valeur logique de l'expression.

Exemples :

```
(x != 0) && (4/x > 3)
```

```
4 > 3 || 1 > 2
```

Exercice

Donnez l'état des variables ligne après ligne :

```

1. int a, b, c;
2. boolean test1, test2, test3, arret;
3. a = 2;
4. b = 3;
5. c = 4;
6. test1 = true;
7. test2 = (b >= a) && (c >= b);
8. test3 = test1 || test2;
9. arret = test3 && (!test2);
10. a += 1;
11. b -= 1;
12. c -= 2;
13. test1 = true;
14. test2 = (b >= a) && (c >= b);
15. test3 = test1 || test2;
16. arret = arret || test2;

```

Bloc

Bloc = instruction composée

Regroupement d'instructions entre { et }

```

{
    x = 3 ;
    y = 7 ;
    z = x * y ;
}
    
```

Visibilité : une entité déclarée dans un bloc est visible à partir de sa déclaration et jusqu'à la fin du bloc

Portée (scope) : exemple

```

    public static void main(String[] args) {
        int x = 0;
        // x est visible
        {
            int y = 0;
            // x et y sont visibles
        }
        // x est visible
        // y est invisible
    }
  
```

Portée de `x`

Portée de `y`

Instruction if

Permet d'exécuter une instruction **si une condition est vérifiée**.

Exemple :

```

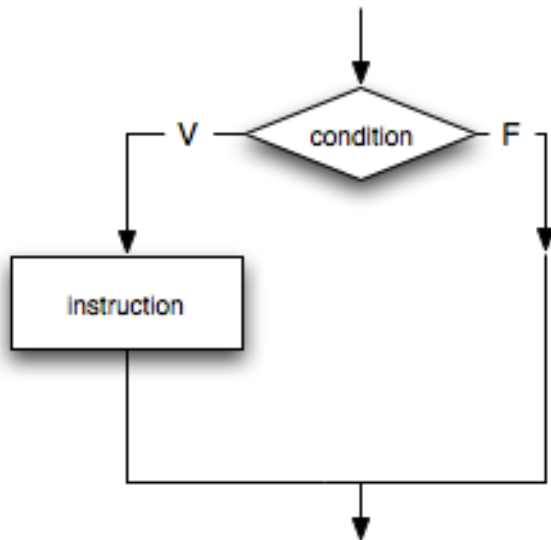
if (cote == 20)
    System.out.println("Bravo");
    
```

Convention : **indentation**

Instruction If

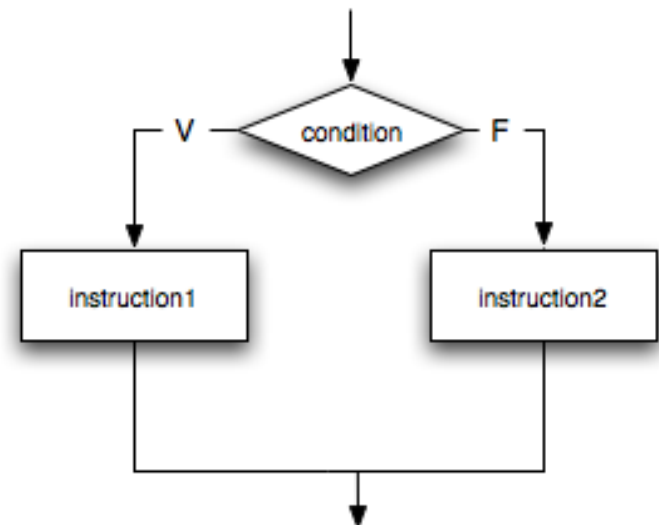
Format 1 :

```
if (condition)  
    instruction
```



Format 2 :

```
if (condition)  
    instruction1  
else  
    instruction2
```



Instruction If

Si le traitement conditionnel est une séquence d'actions, regrouper cette séquence dans un **bloc** :

```

if (i == 20)
{
    System.out.println("Bravo");
    compteur += 1;
}
    
```

Compositions

```

if (condition1)
    System.out.println("cas 1");
else if (condition2)
    System.out.println("cas 2");
else
    System.out.println("cas 3");
  
```

```

if (condition1)
    if (condition2)
        System.out.println("cas a");
    else
        System.out.println("cas b");
  
```


Erreurs courantes

Erreurs

```
if(condition) ;  
    instruction;
```

```
if(condition)  
    instruction1;  
    instruction2;
```

```
if(a = b) ...
```

```
if(a == 2) ...  
else if(a != 2) ...
```

```
if(a < b < c) ...
```

Corrections

```
if(condition)  
    instruction;
```

```
if(condition) {  
    instruction1;  
    instruction2;  
}
```

```
if(a == b)...
```

```
if(a == 2) ...  
else ...
```

```
if(a < b && b < c)
```

Indentation

Par soucis de lisibilité, les instructions conditionnelles doivent être indentées (décalées vers la droite).

Code mal indenté

```
if(a == b)
System.out.println("a==b");
else{
if (b == c)
System.out.println("b==c");
else if(a == c)
System.out.println("a==c");
System.out.println("a!=b");
}
```

Code bien indenté

```
if(a == b)
    System.out.println("a==b");
else {
    if (b == c)
        System.out.println("b==c");
    else if(a == c)
        System.out.println("a==c");
    System.out.println("a!=b");
}
```